

Creación de Servicios Web RESTful con Jakarta REST

Fuente: <https://jakarta.ee/learn/docs/jakartaee-tutorial/current/websvcs/rest/rest.html>

1. ¿Qué son los servicios web RESTful?

Son servicios web ligeros y poco acoplados, es decir, pueden estar **diseñarse y desarrollarse independiente de otros componentes**, y pueden **evolucionar de forma separada** sin afectar el sistema distribuido. Son adecuados para crear un API dirigido a nodos clientes distribuidos en una red o a través de Internet. Asimismo, REST es un estilo arquitectónico de aplicación de tipo cliente-servidor centrado en la transferencia de representaciones de recursos mediante solicitudes y respuestas, por lo general, empleando el protocolo HTTP.

En la arquitectura REST, los datos son considerados recursos y son accedidos a través de una URI (**Uniform Resource Identifiers**), que es un identificador general para nombrar un recurso en Internet. Existen dos tipos de URI: (1) URL y la (2) URN. Su objetivo es identificar un recurso de forma única. En arquitectura REST se utiliza **se usan URLs**, porque RESTful está diseñado para trabajar sobre el protocolo **HTTP**.

En la arquitectura REST, los recursos se manipulan mediante un conjunto fijo de cuatro operaciones:

(1) creación, (2) lectura, (3) actualización y (4) eliminación: (1) PUT, (2) GET, (3) POST y (4) DELETE.

PUT crea un nuevo recurso, que puede eliminarse mediante **DELETE**. **GET** recupera el estado actual de un recurso en alguna representación. **POST** transfiere un nuevo estado a un recurso.

En la arquitectura REST, los recursos se desacoplan de su representación para que se pueda acceder a su contenido en diversos formatos, tales como **HTML, XML, texto plano, JSON** y otros formatos de documento.

2. ¿Qué es Jakarta REST?

Jakarta REST es una API de lenguaje de programación Java diseñada para facilitar el desarrollo de aplicaciones que utilizan la arquitectura REST.

La API REST de Jakarta utiliza anotaciones del lenguaje de programación Java para simplificar el desarrollo de servicios web RESTful. Los desarrolladores programan los archivos de clase del lenguaje de programación Java con anotaciones REST de Jakarta para definir los recursos y las acciones que se pueden realizar en ellos. Las anotaciones REST de Jakarta son anotaciones en tiempo de ejecución; por lo tanto, la **reflexión** en tiempo de ejecución generará las clases auxiliares y los artefactos para el recurso. Un archivo de aplicación de Jakarta EE que contiene clases de recursos REST de Jakarta configurará los recursos, generará las clases auxiliares y los artefactos, y expondrá el recurso a los clientes mediante la implementación del archivo en un servidor de Jakarta EE.

¹ Una técnica para inspeccionar clases, métodos, etc. en tiempo de ejecución) para **leer esas anotaciones** y con base en eso, **crear internamente todo lo necesario** para que esas clases funcionen como servicios REST.

3. Anotaciones de Jakarta EE

Anotación	Descripción
@Path	La anotación @Path es una ruta URI relativa que indica dónde se alojará la clase Java: por ejemplo, /helloworld . También puede incrustar variables en las URI para crear una plantilla de ruta URI. Por ejemplo, podría solicitar el nombre de un usuario y pasarlo a la aplicación como una variable en la URI: /helloworld/{username}
@GET	La anotación @GET designa un método de solicitud y corresponde al método HTTP de nombre similar. El método JAVA anotado con esta etiqueta procesará las solicitudes HTTP GET.

@POST	La anotación @POST designa un método de solicitud y corresponde al método HTTP de nombre similar. El método JAVA anotado con esta etiqueta procesará las solicitudes HTTP POST.
@PUT	La anotación @PUT designa un método de solicitud y corresponde al método HTTP de nombre similar. El método JAVA anotado con este designador procesará las solicitudes HTTP PUT.
@DELETE	La anotación @DELETE designa un método de solicitud y corresponde al método HTTP de nombre similar. El método JAVA anotado con este designador procesará las solicitudes HTTP DELETE.
@PathParam	La anotación @PathParam es tipo de parámetro que se puede extraer para su uso en la clase de recurso. Los parámetros de ruta URI se extraen del URI de la solicitud y sus nombres corresponden a los nombres de las variables de plantilla de ruta URI especificados en la notación @Path a nivel de clase.
@QueryParam	La anotación @QueryParam es un tipo de parámetro que se puede extraer para su uso en la clase de recurso. Los parámetros de consulta se extraen de los parámetros de consulta del URI de solicitud.
@Consumes	La anotación @Consumes se utiliza para especificar los tipos de medios MIME de las representaciones que un recurso puede consumir y que fueron enviadas por el cliente.
@Produces	La anotación @Produces se utiliza para especificar los tipos de medios MIME de las representaciones que un recurso puede producir y enviar al cliente: por ejemplo, "texto/sin formato".
@ApplicationPath	La anotación @ApplicationPath define la asignación de URL para la aplicación. La ruta especificada por el @ApplicationPath es el URI base para todos los URI de recursos especificados por la anotación @Path en la clase de recurso. Solo se puede aplicar @ApplicationPath a una subclase de jakarta.ws.rs.core.Application .

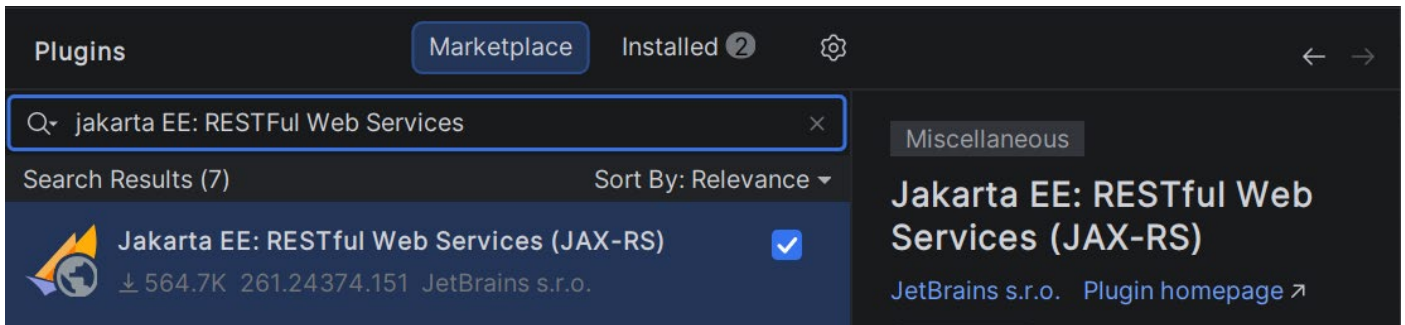
4. Asociación de métodos HTTP con CRUD

En los servicios RESTful, cada operación del **CRUD (Crear, Leer, Actualizar, Eliminar)** se asocia con un método HTTP estándar. A continuación, se presenta la relación:

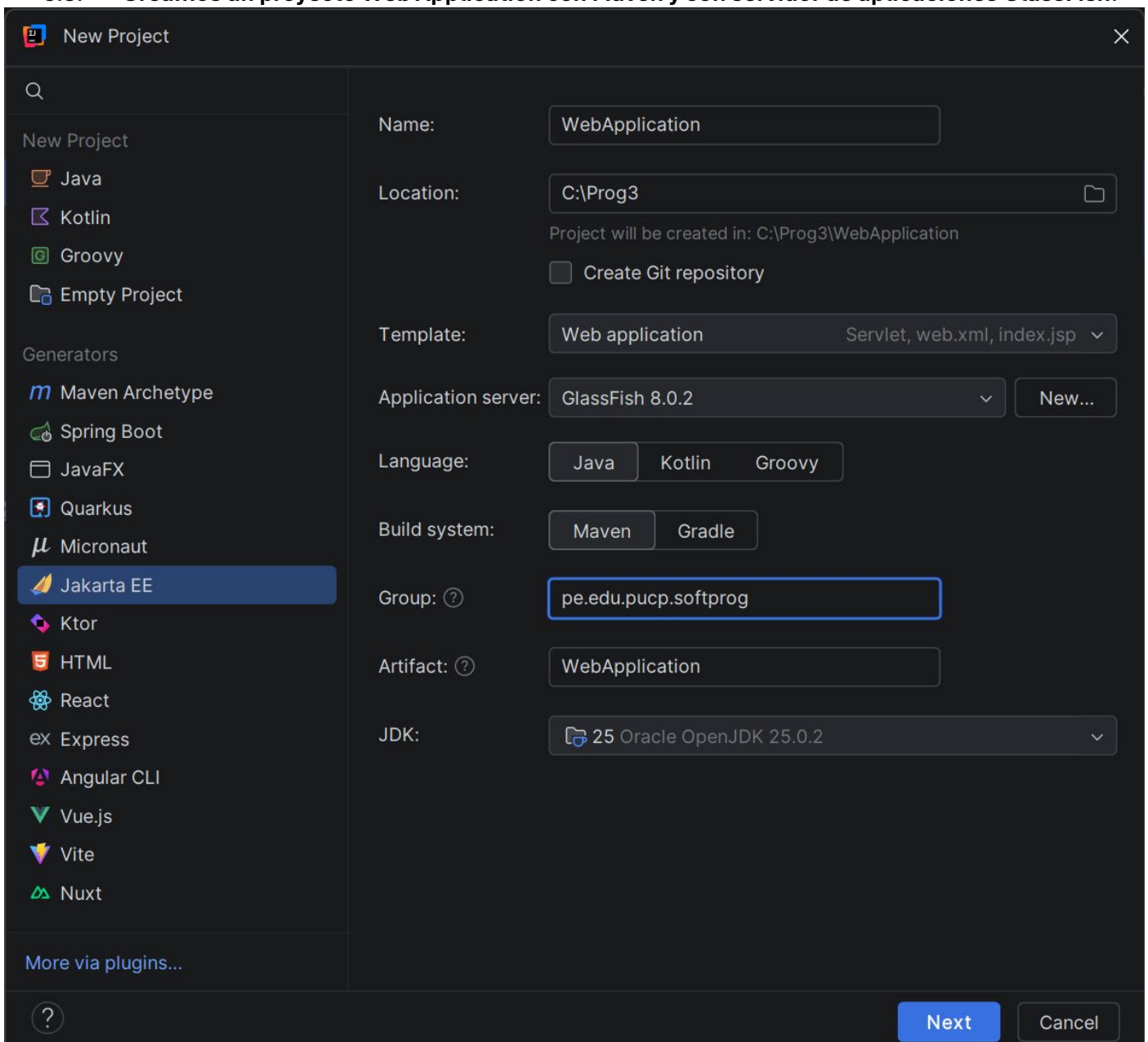
Operación CRUD	Método HTTP	Descripción
CREATE	POST	Crear, Insertar, Registrar (Para enviar datos al servidor para crear un nuevo recurso).
READ (TODOS)	GET	Listar todos, Listar por criterio (Para solicitar la lista completa de recursos).
READ (UNO)	GET	Obtener por id, Obtener por criterio (Para solicitar un recurso específico).
UPDATE	PUT	Actualizar, Modificar (Para enviar datos al servidor para modificar un recurso existente).
DELETE	DELETE	Eliminar (Para solicitar la eliminación de un recurso específico, normalmente indicando su ID)

5. Ejemplo Básico de Implementación

- 5.1. Nos aseguramos de tener configurado el Glassfish en el IDE, así como su plugin.
- 5.2. Ingresamos File, Settings, Plugins e instalamos el plugin:
Jakarta EE: RESTful Web Services (JAX-RS)

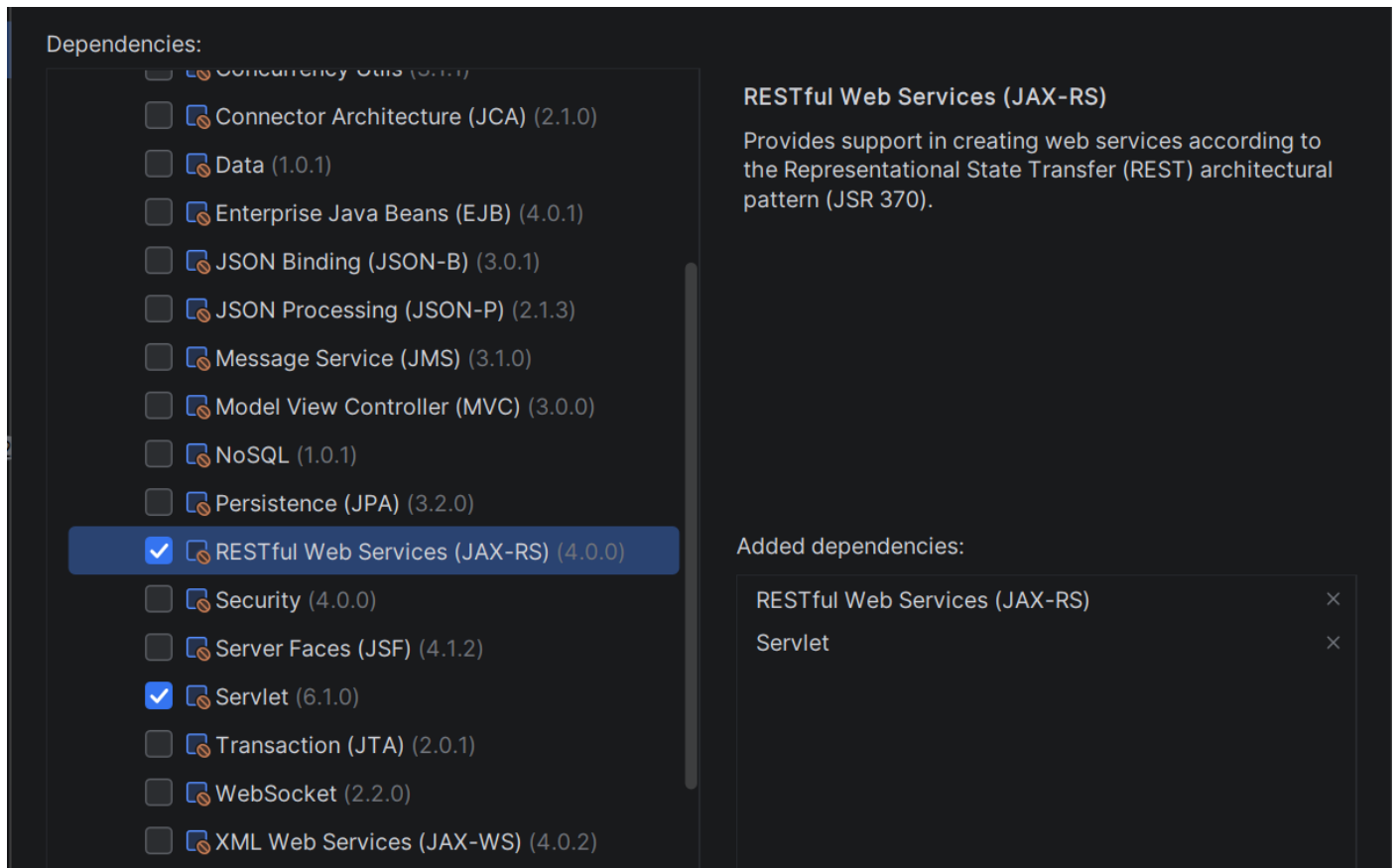


- 5.3. Creamos un proyecto Web Application con Maven y con servidor de aplicaciones GlassFish.



Le colocamos un nombre al proyecto. En este caso se está colocando como nombre: "WebApplication".

5.4. Elegimos la dependencia “RESTful Web Services (JAX-RS).



Dependencies:

- ☐ Concurrency Utils (3.1.1)
- ☐ Connector Architecture (JCA) (2.1.0)
- ☐ Data (1.0.1)
- ☐ Enterprise Java Beans (EJB) (4.0.1)
- ☐ JSON Binding (JSON-B) (3.0.1)
- ☐ JSON Processing (JSON-P) (2.1.3)
- ☐ Message Service (JMS) (3.1.0)
- ☐ Model View Controller (MVC) (3.0.0)
- ☐ NoSQL (1.0.1)
- ☐ Persistence (JPA) (3.2.0)
- ☒ RESTful Web Services (JAX-RS) (4.0.0)
- ☐ Security (4.0.0)
- ☐ Server Faces (JSF) (4.1.2)
- ☒ Servlet (6.1.0)
- ☐ Transaction (JTA) (2.0.1)
- ☐ WebSocket (2.2.0)
- ☐ XML Web Services (JAX-WS) (4.0.2)

RESTful Web Services (JAX-RS)

Provides support in creating web services according to the Representational State Transfer (REST) architectural pattern (JSR 370).

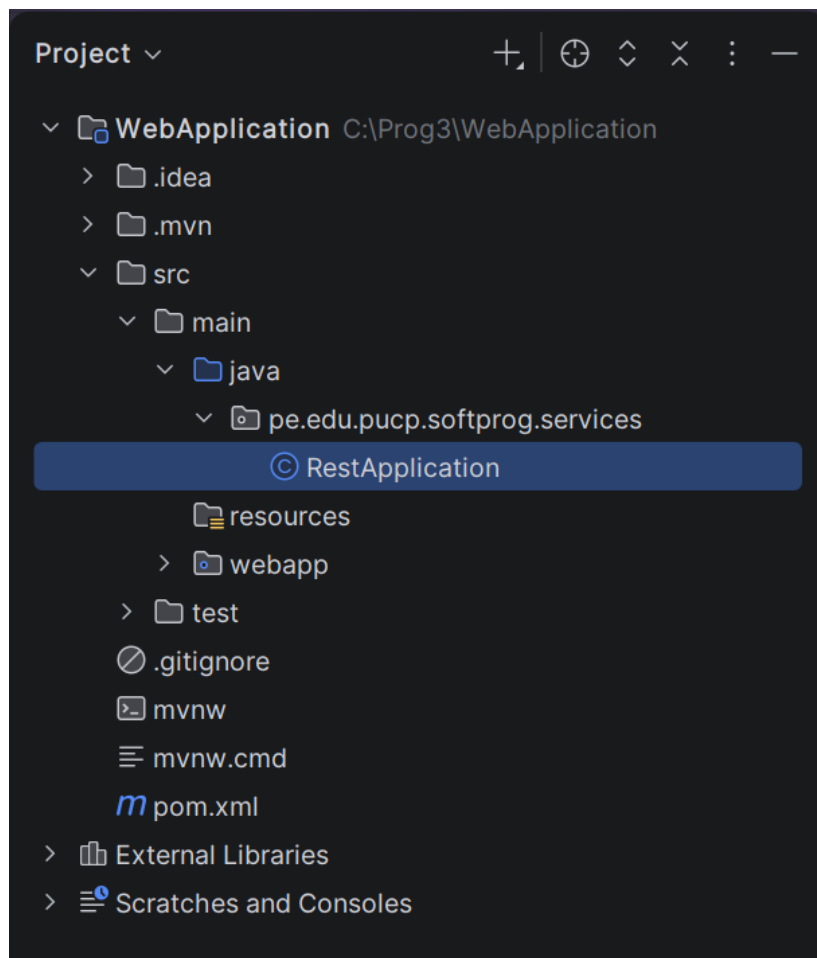
Added dependencies:

- RESTful Web Services (JAX-RS) ×
- Servlet ×

5.5. Una vez creado el proyecto web, verificamos en el pom.xml que se encuentre correctamente configurada la versión de JDK para el proyecto web. De no ser así, modificamos a 25.

```
13     <properties>
14         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15         <maven.compiler.target>25</maven.compiler.target>
16         <maven.compiler.source>25</maven.compiler.source>
17         <junit.version>5.13.2</junit.version>
18     </properties>
```

5.6. Creamos un paquete y una clase que permita el escaneo de servicios RESTful.



```
package pe.edu.pucp.softprog.services;
import jakarta.ws.rs.ApplicationPath;
import jakarta.ws.rs.core.Application;

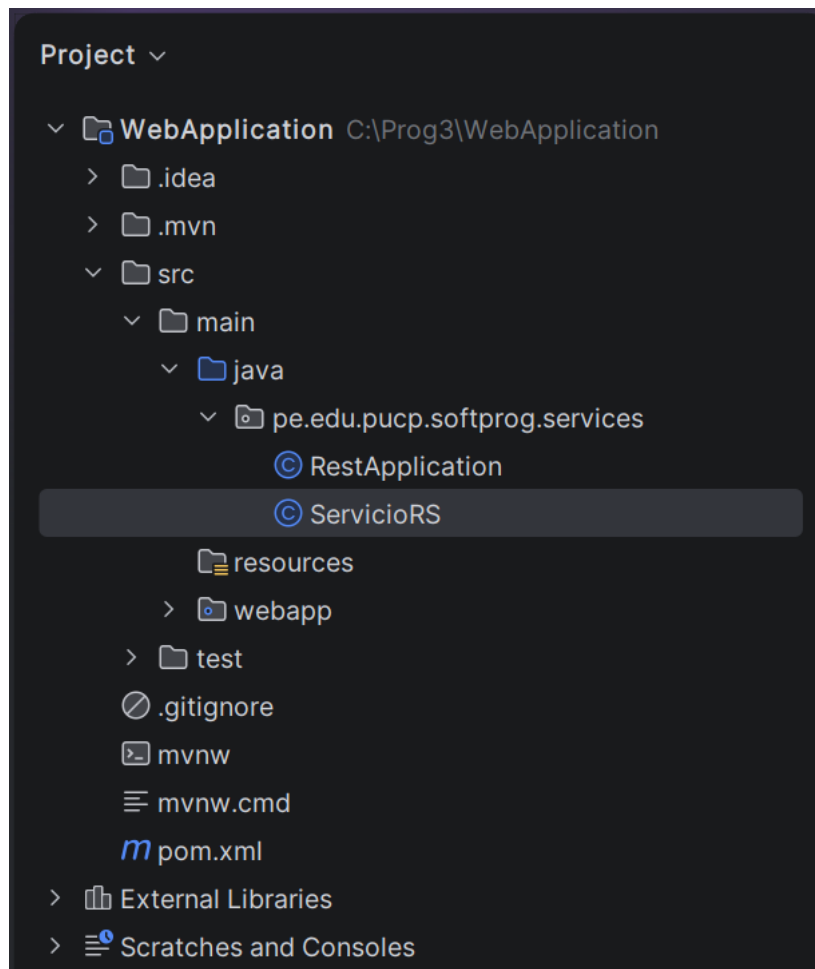
@ApplicationPath("webresources")
public class RestApplication extends Application {
    // Esta clase activa el escaneo de recursos REST
}
```

- `@ApplicationPath("webresources")`: Es una anotación que indica que todos tus servicios REST estarán disponibles bajo este prefijo de ruta. Este prefijo de ruta puede variar, el programador lo elige.
- El nombre de la clase puede variar, el programador lo elige.
- Extiende de la clase `Application`, lo que permite indicarle al servidor que esta clase gestiona la configuración de los recursos REST.

Con esta programación, el servidor realiza lo siguiente:

- Escanea esta clase.
- Detecta `@ApplicationPath("webresources")`.
- Escanea todas las clases anotadas con `@Path(...)` dentro del proyecto.
- Crea automáticamente los "endpoints" (puntos de acceso) REST usando la base `webresources`.

5.7. Creamos una clase que representará el servicio REST



```
package pe.edu.pucp.softprog.services;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.Consumes;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.QueryParam;

@Path("ServicioRS")
public class ServicioRS {

    @GET
    @Produces("text/html")
    public String getHtml(@QueryParam("nombre") String nombre) {
        if (nombre == null || nombre.isBlank()) {
            nombre = "Invitado";
        }
        return "<html lang=\"es\">" +
            "<head><title>Página de Saludo</title></head>" +
            "<body><h1>Hola " + nombre + "!</h1></body>" +
            "</html>";
    }
}
```


- **@Path("ServicioWS")**: Declara la clase como un recurso REST, accesible mediante la ruta ServicioWS.
- **@GET**: Indica que el método responderá a solicitudes HTTP GET, que normalmente se usan para obtener datos.
- **@Produces("text/html")**: Indica que el método devolverá contenido HTML. El navegador podrá interpretar directamente lo que devuelve este método.
- **@QueryParam("nombre")**: Indica que el valor del parámetro nombre se tomará desde la URL. Por ejemplo: `?nombre=Laura`

5.8. Ejecutamos la aplicación web.

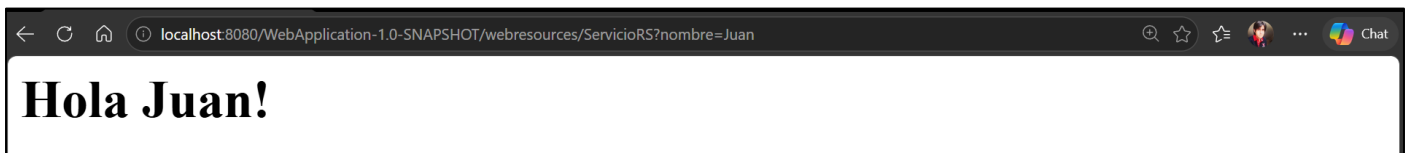
5.9. Abrimos un navegador para probar el servicio.

Escribimos la URL:

[http://localhost:8080/\[ContextPathAplicacion\]/\[PrefijoRutaDeLosServicios\]/\[NombreServicio\]](http://localhost:8080/[ContextPathAplicacion]/[PrefijoRutaDeLosServicios]/[NombreServicio])

En este ejemplo sería:

<http://localhost:8080/WebApplication-1.0-SNAPSHOT/webresources/ServicioRS?nombre=Juan>



- 5.10. Dentro de la sección de "build" en el archivo pom.xml podemos agregar la siguiente línea para que el context path sea únicamente WebApplication y se pueda acceder de esa manera.

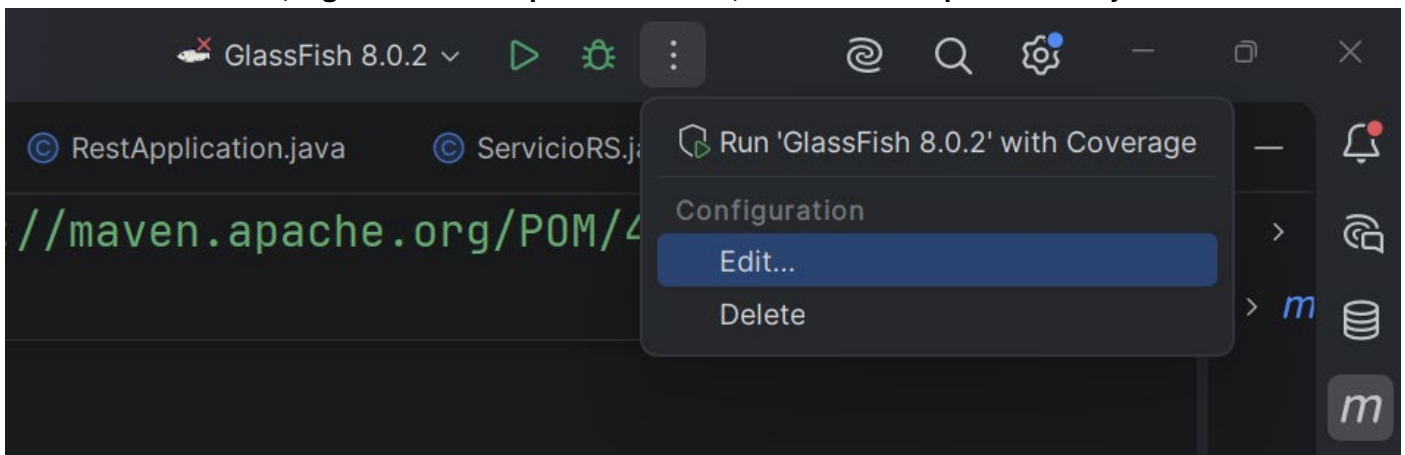
```
<finalName>WebApplication</finalName>
```

5.11. Quedaría de la siguiente manera:

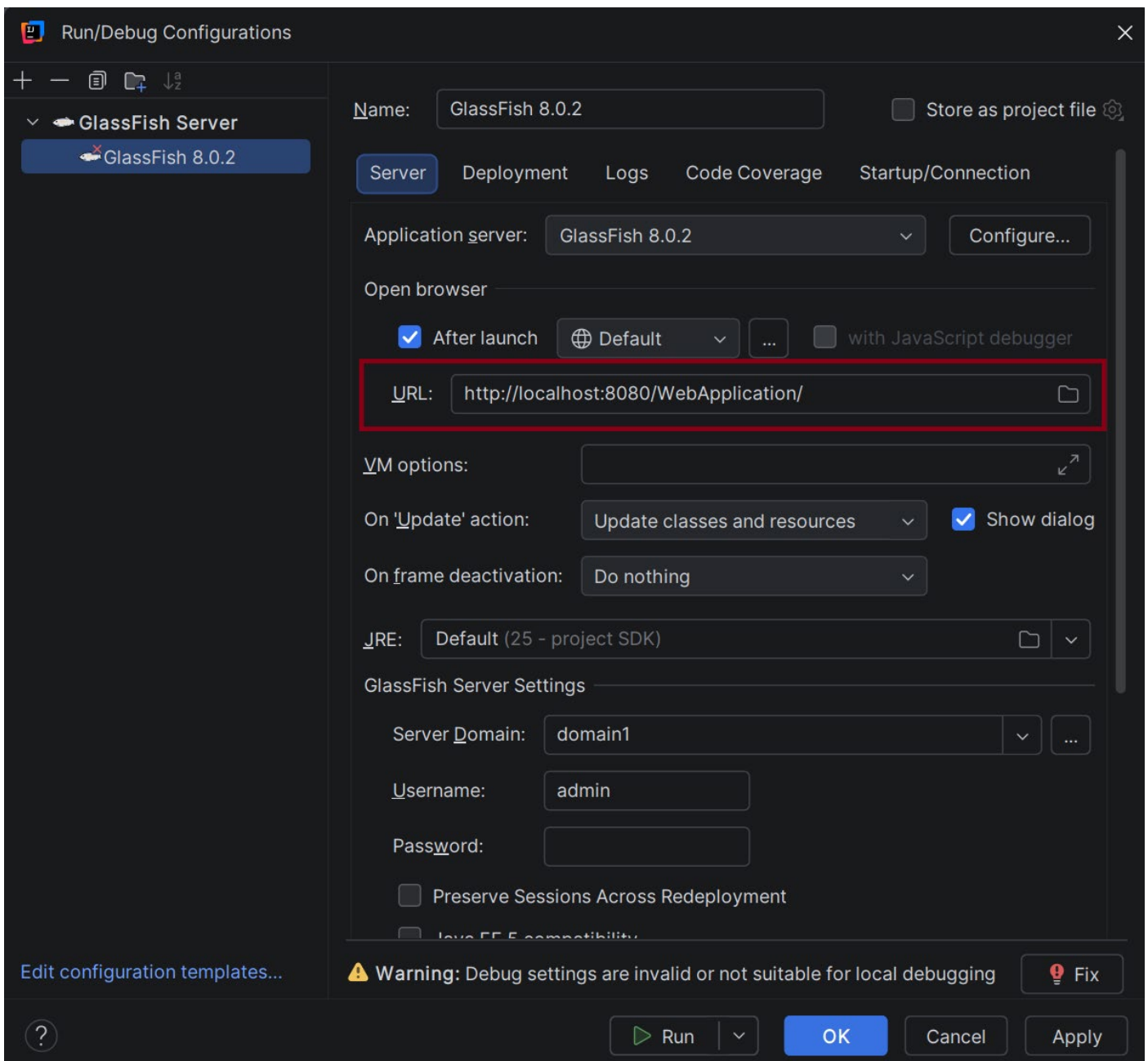
```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  <dependencies>
  </dependencies>

  <build>
    <finalName>WebApplication</finalName>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-war-plugin</artifactId>
        <version>3.4.0</version>
      </plugin>
    </plugins>
  </build>
</project>
```

5.12. Asimismo, ingresamos a la opción de “Edit”, dentro de las opciones de Ejecución



5.13. Cambiamos la URL:



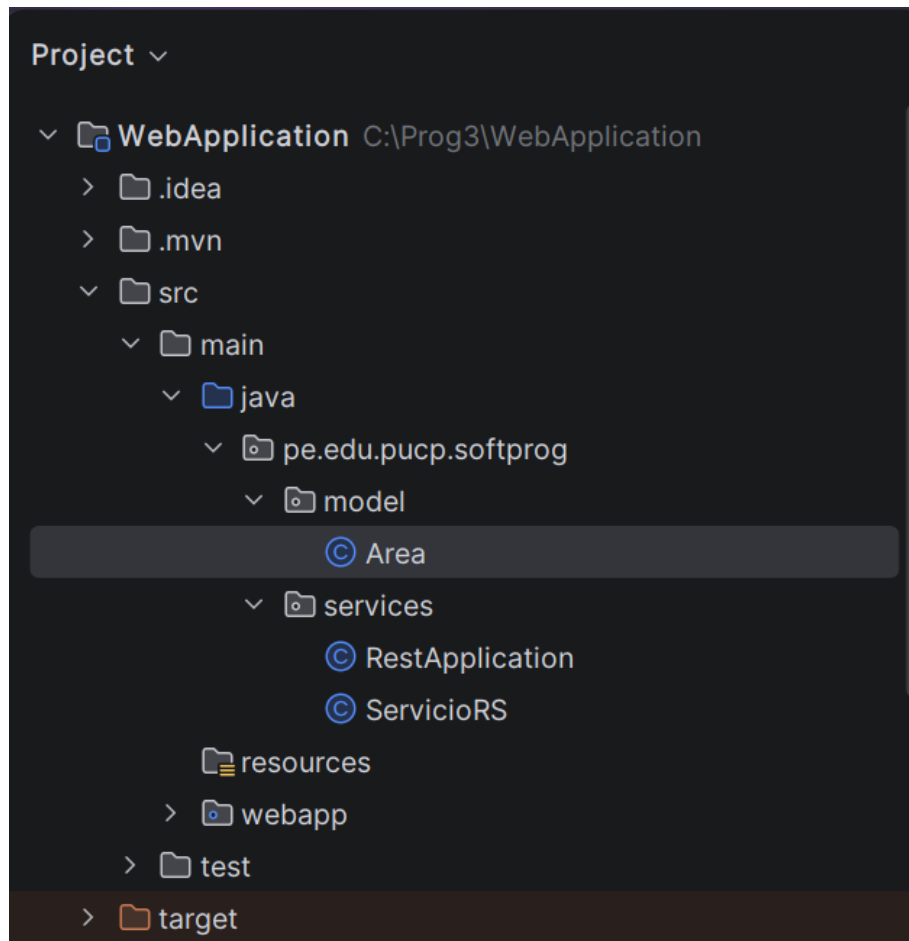
5.14. De esa manera, la próxima vez que ejecutemos, ya sería posible acceder con la siguiente URL:

<http://localhost:8080/WebApplication/webresources/ServicioRS?nombre=Juan>

6. Ejemplo con devolución en formato JSON

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos entre programas, fácil de leer y entender. Es ampliamente utilizado en aplicaciones web para enviar y recibir información entre el cliente y el servidor, especialmente en servicios web RESTful.

6.1. Creamos una clase del dominio.



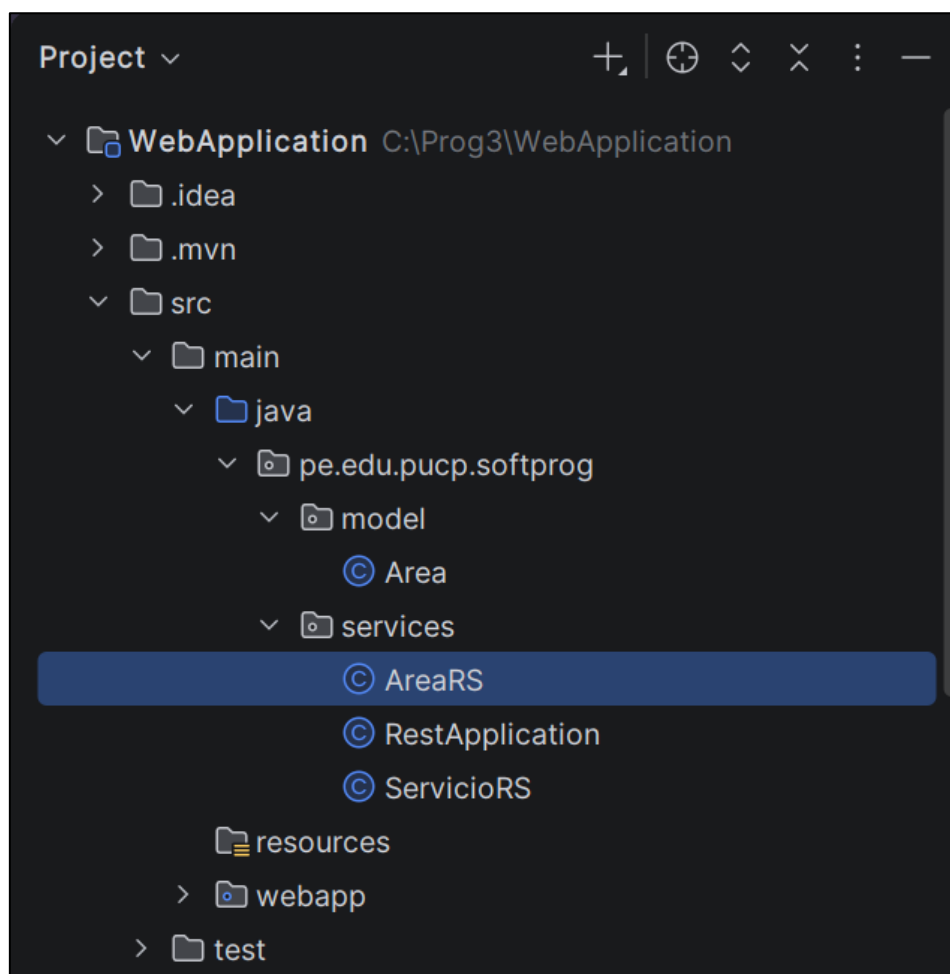
```
package pe.edu.pucp.softprog.model;
public class Area {
    private int idArea;
    private String nombre;
    private boolean activa;

    public Area(int idArea, String nombre, boolean activa){
        this.idArea = idArea;
        this.nombre = nombre;
        this.activa = activa;
    }
    public Area(){}

    public int getIdArea() {
        return idArea;
    }
}
```

```
public void setIdArea(int idArea) {  
    this.idArea = idArea;  
}  
  
public String getNombre() {  
    return nombre;  
}  
  
public void setNombre(String nombre) {  
    this.nombre = nombre;  
}  
  
public boolean isActiva() {  
    return activa;  
}  
  
public void setActiva(boolean activa) {  
    this.activa = activa;  
}  
}
```

6.2. Creamos la clase del servicio



```
package pe.edu.pucp.softprog.services;

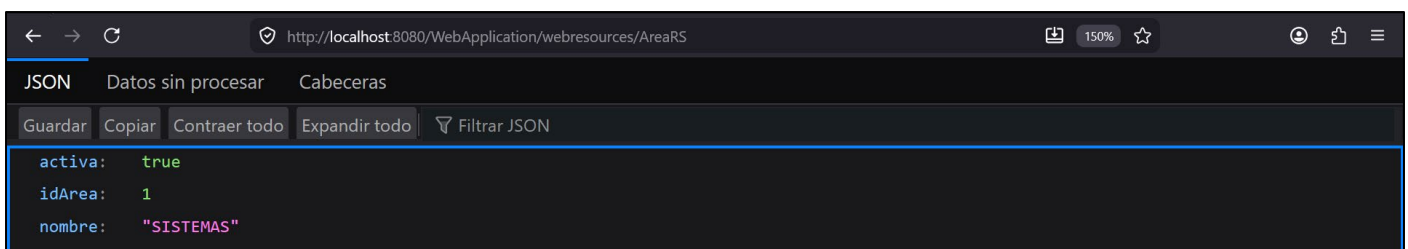
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;
import pe.edu.pucp.softprog.model.Area;

@Path("AreaRS")
public class AreaRS {

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public Area obtenerArea() {
        Area area = new Area();
        area.setIdArea(1);
        area.setNombre("SISTEMAS");
        area.setActiva(true);
        return area;
    }
}
```

- **@Produces(MediaType.APPLICATION_JSON)**: Indica que la respuesta será JSON.

Si accedemos a: <http://localhost:8080/WebApplication/webresources/AreaRS>



7. Ejemplo con todo el CRUD de una entidad

- Se utilizará una lista `ArrayList<Area>` para simular una pequeña base de datos.
- El servicio a crear permitirá obtener todas las áreas, obtener una por ID, insertar, modificar y eliminar.
- Se utilizará la misma clase del dominio **Area** que en el ejemplo anterior.

7.1. Creando el servicio RESTFul completo

```
package pe.edu.pucp.softprog.services;

import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;
import java.util.ArrayList;
import java.util.List;
import pe.edu.pucp.softprog.model.Area;

@Path("AreaRS")
@Produces(MediaType.APPLICATION_JSON) // Todas las respuestas son JSON
@Consumes(MediaType.APPLICATION_JSON) // Todas las peticiones esperan JSON
public class AreaRS {

    // Simulamos una base de datos en memoria
    private static List<Area> listaAreas = new ArrayList<>();
    private static int contadorId = 1;

    static {
        // Áreas iniciales
        listaAreas.add(new Area(contadorId++, "SISTEMAS", true));
        listaAreas.add(new Area(contadorId++, "CONTABILIDAD", true));
    }

    // Obtener todas las áreas
    @GET
    public List<Area> listarAreas() {
        return listaAreas;
    }
}
```

```
// Obtener un área por su ID
@GET
@Path("/{id}")
public Response obtenerAreaPorId(@PathParam("id") int id) {
    for (Area a : listaAreas) {
        if (a.getIdArea() == id) {
            return Response.ok(a).build(); // JSON
        }
    }
    return Response.status(Response.Status.NOT_FOUND).build();
}

// Crear una nueva área
@POST
public Response crearArea(Area nuevaArea) {
    nuevaArea.setIdArea(contadorId++);
    listaAreas.add(nuevaArea);
    return Response.status(Response.Status.CREATED).entity(nuevaArea).build(); //
JSON
}

// Modificar un área existente
@PUT
@Path("/{id}")
public Response actualizarArea(@PathParam("id") int id, Area areaActualizada) {
    for (int i = 0; i < listaAreas.size(); i++) {
        if (listaAreas.get(i).getIdArea() == id) {
            areaActualizada.setIdArea(id); // No cambiar el ID
            listaAreas.set(i, areaActualizada);
            return Response.ok(areaActualizada).build(); // JSON
        }
    }
    return Response.status(Response.Status.NOT_FOUND).build();
}

// Eliminar un área por ID
@DELETE
@Path("/{id}")
public Response eliminarArea(@PathParam("id") int id) {
    for (Area a : listaAreas) {
        if (a.getIdArea() == id) {
            listaAreas.remove(a);
            return Response.noContent().build(); // 204 sin cuerpo
        }
    }
    return Response.status(Response.Status.NOT_FOUND).build();
}
}
```

7.2. Probando el servicio RESTFul completo

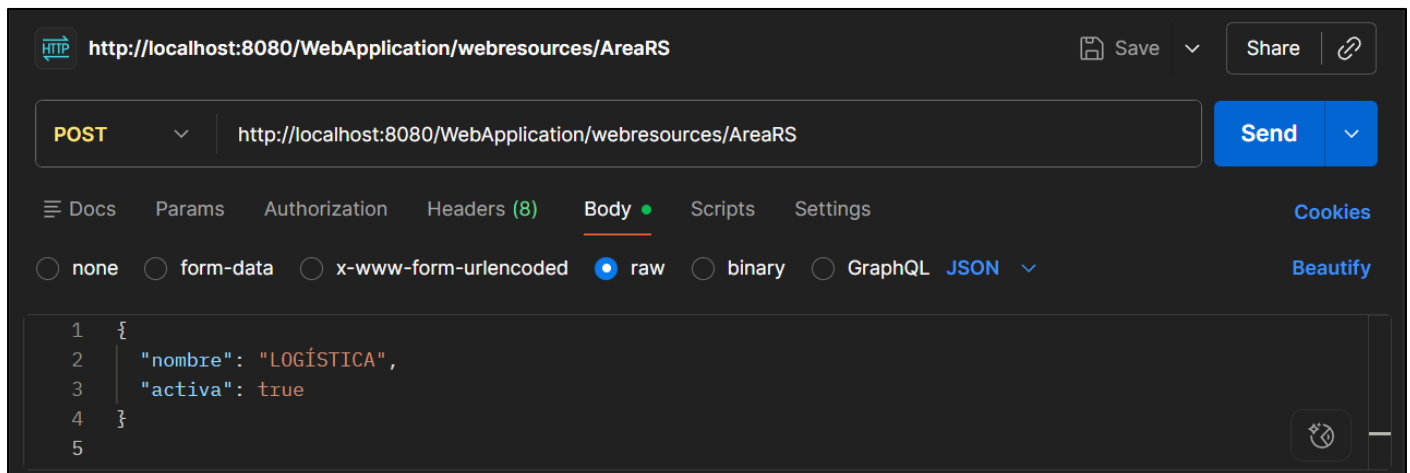
Para probar los métodos del servicio AreaWS que devuelve y recibe datos en formato JSON, podemos usar una herramienta llamada **Postman**. Postman es una aplicación gratuita y multiplataforma que permite realizar peticiones HTTP (GET, POST, PUT, DELETE, etc.) y analizar las respuestas del servidor.

- **CREAR UN AREA (POST)**

URL: <http://localhost:8080/WebApplication/webresources/AreaRS>

Body (JSON):

```
{
  "nombre": "LOGÍSTICA",
  "activa": true
}
```



- **OBTENER UN AREA POR ID (GET)**

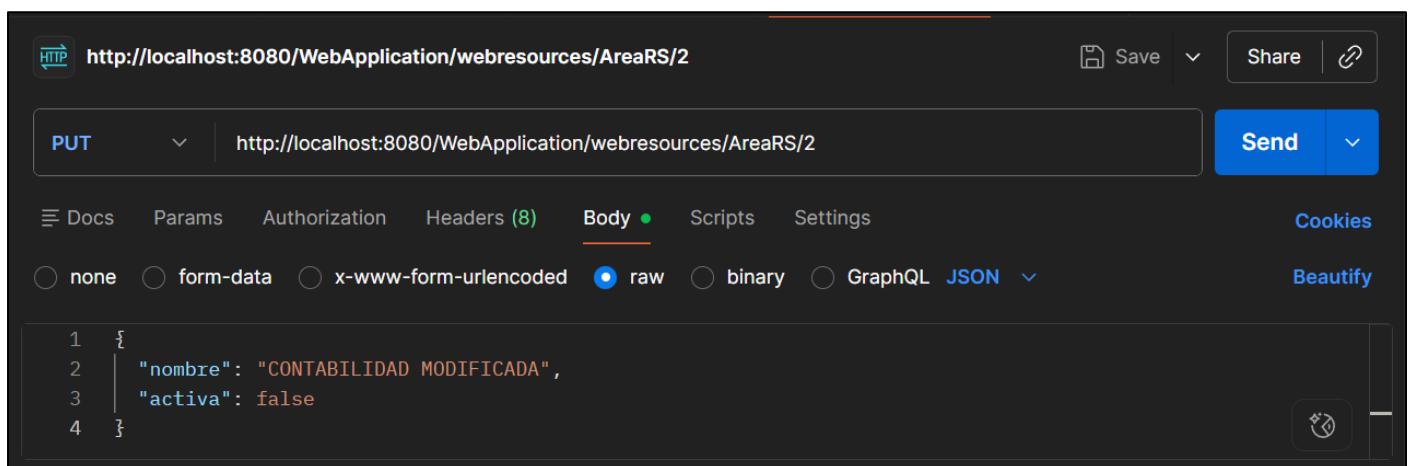
URL: <http://localhost:8080/WebApplication/webresources/AreaRS/1>

- **MODIFICAR AREA (PUT)**

URL: <http://localhost:8080/WebApplication/webresources/AreaRS/2>

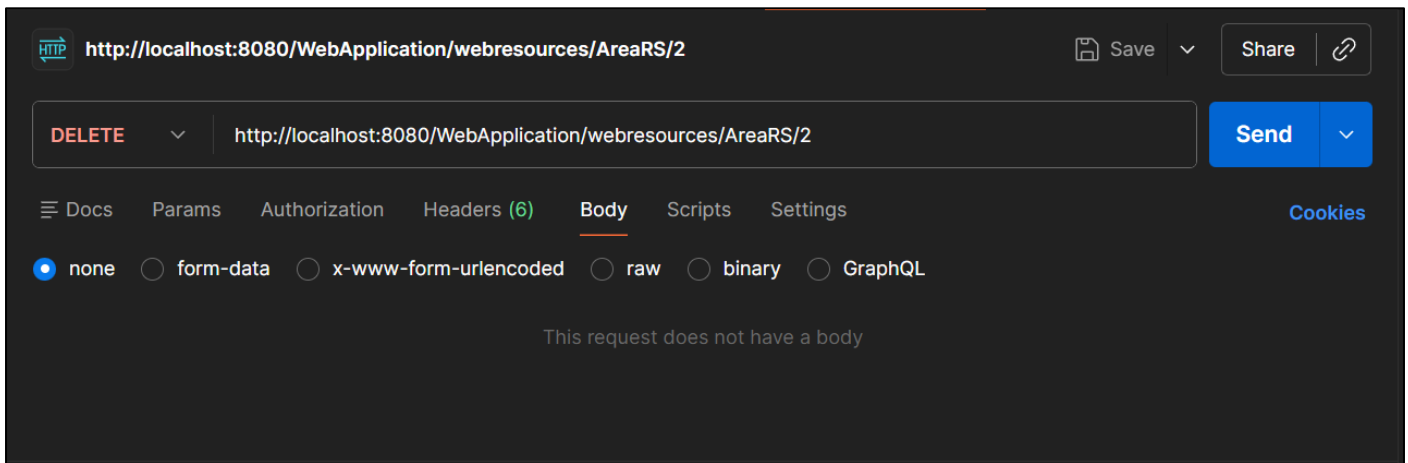
Body (JSON):

```
{
  "nombre": "CONTABILIDAD MODIFICADA",
  "activa": false
}
```



- **ELIMINAR AREA (DELETE)**

URL: <http://localhost:8080/WebApplication/webresources/AreaRS/2>



- **OBTENER TODAS LAS AREAS (GET)**

URL: <http://localhost:8080/WebApplication/webresources/AreaRS>

8. Consumiendo servicios RESTful en JAVA

Para ello, utilizaremos la librería "jackson-databind".

8.1. Agregamos la dependencia al proyecto.

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
  <version>2.19.0</version>
</dependency>
```

8.2. Realizamos la codificación de un cliente en JAVA que **LISTA TODAS LAS AREAS.**

```
package pe.edu.pucp.softprog.client;

import com.fasterxml.jackson.core.type.TypeReference;
import com.fasterxml.jackson.databind.ObjectMapper;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.util.ArrayList;

public class Principal {

    public static void main(String[] args) throws Exception{
        String url = "http://localhost:8080/WebApplication/webresources/AreaWS";
        HttpClient client = HttpClient.newHttpClient();
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(url))
            .GET()
            .build();
```

```
        HttpResponse<String> response = client.send(request,
        HttpResponse.BodyHandlers.ofString());
        String json = response.body();
        ObjectMapper mapper = new ObjectMapper();
        ArrayList<Area> listaAreas = mapper.readValue(json, new
        TypeReference<ArrayList<Area>>() {});

        for (Area area : listaAreas) {
            System.out.println(area.getIdArea() + " - " + area.getNombre() + " -
        Activa: " + area.isActiva());
        }
    }
}
```

8.3. Realizamos la codificación de un cliente que **INSERTA UN AREA.**

```
package pe.edu.pucp.softprog.client;

import com.fasterxml.jackson.databind.ObjectMapper;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class Principal {
    public static void main(String[] args) throws Exception{
        String url = "http://localhost:8080/WebApplication/webresources/AreaWS";

        // Creamos un objeto Area
        Area nuevaArea = new Area();
        nuevaArea.setNombre("RECURSOS HUMANOS");
        nuevaArea.setActiva(true);

        // Serializamos a JSON
        ObjectMapper mapper = new ObjectMapper();
        String jsonRequest = mapper.writeValueAsString(nuevaArea);

        // Creamos la solicitud POST
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(url))
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(jsonRequest))
            .build();

        // Creamos el cliente HTTP
        HttpClient client = HttpClient.newHttpClient();

        // Enviamos la solicitud
        HttpResponse<String> response = client.send(request,
        HttpResponse.BodyHandlers.ofString());

        // Mostramos respuesta
        System.out.println("Código de respuesta: " + response.statusCode());
    }
}
```

```
System.out.println("Respuesta del servidor:");
System.out.println(response.body());

// Si el servidor devuelve el Area insertada, puedes deserializarla:
Area areaRespuesta = mapper.readValue(response.body(), Area.class);
System.out.println("Área creada: " + areaRespuesta.getIdArea() + " - " +
areaRespuesta.getNombre());
    }
}
```

8.4. Realizamos la codificación de un cliente que **MODIFICA UN AREA**.

```
package pe.edu.pucp.softprog.client;

import com.fasterxml.jackson.databind.ObjectMapper;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class PrincipalModificar {
    public static void main(String[] args) throws Exception{
        int idAreaAModificar = 1; // El ID del área a actualizar
        String url = "http://localhost:8080/WebApplication/webresources/AreaWS/" +
idAreaAModificar;

        // Creamos el objeto con los nuevos datos
        Area areaModificada = new Area();
        areaModificada.setNombre("LOGÍSTICA");
        areaModificada.setActiva(false);

        // Convertir el objeto a JSON
        ObjectMapper mapper = new ObjectMapper();
        String jsonRequest = mapper.writeValueAsString(areaModificada);

        // Crear la solicitud PUT
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(url))
            .header("Content-Type", "application/json")
            .PUT(HttpRequest.BodyPublishers.ofString(jsonRequest))
            .build();

        // Crear cliente y enviar
        HttpClient client = HttpClient.newHttpClient();
        HttpResponse<String> response = client.send(request,
        HttpResponse.BodyHandlers.ofString());

        // Mostrar respuesta
        System.out.println("Código de respuesta: " + response.statusCode());
        System.out.println("Respuesta del servidor:");
        System.out.println(response.body());

        // Si el servidor devuelve el área actualizada, la podemos deserializar
        Area areaRespuesta = mapper.readValue(response.body(), Area.class);
    }
}
```

```
        System.out.println("Área actualizada: " + areaRespuesta.getIdArea() + " - " +  
        areaRespuesta.getNombre());  
    }  
}
```

8.5. Realizamos la codificación de un cliente que **ELIMINA UN AREA.**

```
package pe.edu.pucp.softprog.client;  
  
import java.net.URI;  
import java.net.http.HttpClient;  
import java.net.http.HttpRequest;  
import java.net.http.HttpResponse;  
  
public class PrincipalEliminar {  
    public static void main(String[] args) throws Exception{  
        int idAreaAEliminar = 1;  
        String url = "http://localhost:8080/MiApp/webresources/AreaWS/" +  
idAreaAEliminar;  
  
        // Crear solicitud DELETE  
        HttpRequest request = HttpRequest.newBuilder()  
            .uri(URI.create(url))  
            .DELETE()  
            .build();  
  
        // Enviar solicitud  
        HttpClient client = HttpClient.newHttpClient();  
        HttpResponse<String> response = client.send(request,  
        HttpResponse.BodyHandlers.ofString());  
  
        // Mostrar respuesta  
        System.out.println("Código de respuesta: " + response.statusCode());  
        System.out.println("Respuesta del servidor:");  
        System.out.println(response.body());  
    }  
}
```

8.6. Realizamos la codificación de un cliente que **OBTIENE UN AREA POR ID.**

```
package pe.edu.pucp.softprog.client;  
  
import com.fasterxml.jackson.databind.ObjectMapper;  
import java.net.URI;  
import java.net.http.HttpClient;  
import java.net.http.HttpRequest;  
import java.net.http.HttpResponse;  
  
public class PrincipalObtenerPorID {  
    public static void main(String[] args) throws Exception{  
        int idArea = 2;  
        String url = "http://localhost:8080/WebApplication/webresources/AreaWS/" +  
idArea;
```

```
// Crear solicitud GET
HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create(url))
    .header("Accept", "application/json")
    .GET()
    .build();

// Crear cliente y enviar
HttpClient client = HttpClient.newHttpClient();
HttpResponse<String> response = client.send(request,
    HttpResponse.BodyHandlers.ofString());

// Verificar respuesta
System.out.println("Código de respuesta: " + response.statusCode());
System.out.println("JSON recibido: " + response.body());

// Convertir JSON a objeto Area
ObjectMapper mapper = new ObjectMapper();
Area area = mapper.readValue(response.body(), Area.class);

System.out.println("Área recibida:");
System.out.println("ID: " + area.getIdArea());
System.out.println("Nombre: " + area.getNombre());
System.out.println("Activa: " + area.isActiva());
}
}
```

9. Consumiendo servicios RESTful en C#

Instalar: Newtonsoft.Json

9.1. Creamos el modelo indicando la propiedad Json en cada propiedad

```
public class Area
{
    private int idArea;
    private string nombre;
    private bool activa;

    [JsonProperty("idArea")]
    public int IdArea { get => idArea; set => idArea = value; }

    [JsonProperty("nombre")]
    public string Nombre { get => nombre; set => nombre = value; }

    [JsonProperty("activa")]
    public bool Activa { get => activa; set => activa = value; }
}
```

9.2. Listando todas las áreas

```
private const string BASE_URL = "http://localhost:8080/SoftProgRS/webresources/AreaRS";
private static readonly HttpClient http = new HttpClient();
static void Main(string[] args)
{
    HttpWebRequest req = (HttpWebRequest)WebRequest.Create($"{BASE_URL}/listarTodas");
    req.Method = "GET";
}
```

```
req.Accept = "application/json";
req.Timeout = 30000;

HttpWebResponse resp = (HttpWebResponse)req.GetResponse();
StreamReader sr = new StreamReader(resp.GetResponseStream());
string json = sr.ReadToEnd();
BindingList<Area> areas =
    JsonConvert.DeserializeObject<BindingList<Area>>(json) ?? new BindingList<Area>();

foreach (Area a in areas)
    System.Console.WriteLine(a.IdArea + ". " + a.Nombre);
}
```

9.3. Insertando un área

```
internal class Program
{
    private const string BASE_URL = "http://localhost:8080/SoftProgRS/webresources/AreaRS";
    private static readonly HttpClient http = new HttpClient();
    static void Main(string[] args)
    {
        HttpWebRequest req = (HttpWebRequest)WebRequest.Create($"{BASE_URL}/insertarArea");
        req.Method = "POST";
        req.Accept = "application/json";
        req.ContentType = "application/json";
        req.Timeout = 30000;

        Area area = new Area();
        area.Nombre = "RECURSOS HUMANOS";

        string jsonBody = JsonConvert.SerializeObject(area);

        StreamWriter sw = new StreamWriter(req.GetRequestStream());
        sw.Write(jsonBody);
        sw.Flush();
        sw.Close();

        HttpWebResponse resp = (HttpWebResponse)req.GetResponse();
        StreamReader sr = new StreamReader(resp.GetResponseStream());
        string json = sr.ReadToEnd();
        int resultado;

        try
        {
            resultado = JsonConvert.DeserializeObject<int>(json);
        }
        catch
        {
            resultado = int.Parse(json);
        }
        Console.WriteLine("Resultado de inserción: " + resultado);
        if(resultado!=0)
            Console.WriteLine("Inserción finalizada correctamente.");
    }
}
```